FastAPI Reference

`HTTPConnection` class

When you want to define dependencies that should be compatible with both HTTP and WebSockets, you can define a parameter that takes an `HTTPConnection` instead of a `Request` or a `WebSocket`.

You can import it from `fastapi.requests`:

```
from fastapi.requests import HTTPConnection
```

class fastapi.requests.HTTPConnection

```
HTTPConnection(scope, receive=None)
```

Bases: `Mapping[str, Any]`, `Generic[StateT]`

A base class for incoming HTTP connections, that is used to provide any functionality that is common to both `Request` and `WebSocket`.

Source code in `starlette/requests.py`

```
def __init__(self, scope: Scope, receive: Receive | None = None) -> None:
    assert scope["type"] in ("http", "websocket")
    self.scope = scope
```

attr scope `instance-attribute`

```
scope = scope
```

attr app `property`

```
app
```

attr url `property`

```
url
```

attr base_url `property`

```
base_url
```

attr headers `property`

```
headers
```

Ask AI

attr query_params `property`

```
query_params
```

attr path_params `property`

```
path_params
```

attr cookies `property`

```
cookies
```

attr client `property`

```
client
```

attr session `property`

```
session
```

attr auth `property`

```
auth
```

attr user `property`

```
user
```

attr state `property`

```
state
```

meth url_for

```
url_for(name, /, **path_params)
```

Source code in `starlette/requests.py`

```
def url_for(self, name: str, /, **path_params: Any) -> URL:
    url_path_provider: Router | Starlette | None = self.scope.get("router") or
self.scope.get("app")
    if url_path_provider is None:
        raise RuntimeError("The `url_for` method can only be used inside a Starlette application
or with a router.")
    url_path = url_path_provider.url_path_for(name, **path_params)
    return url_path.make_absolute_url(base_url=self.base_url)
```